WEEK 02

Comparisons and branches

Python foundations

Comparisons and branches

Values, conditions and complete decisions

Compare values

A comparison produces a Boolean. >= includes the boundary.

```
print(10 > 10)   # False
print(10 >= 10)  # True
```

Combine conditions

and needs both; or needs at least one; not reverses a Boolean.

```
print(True and False)  # False
print(not False)       # True
```

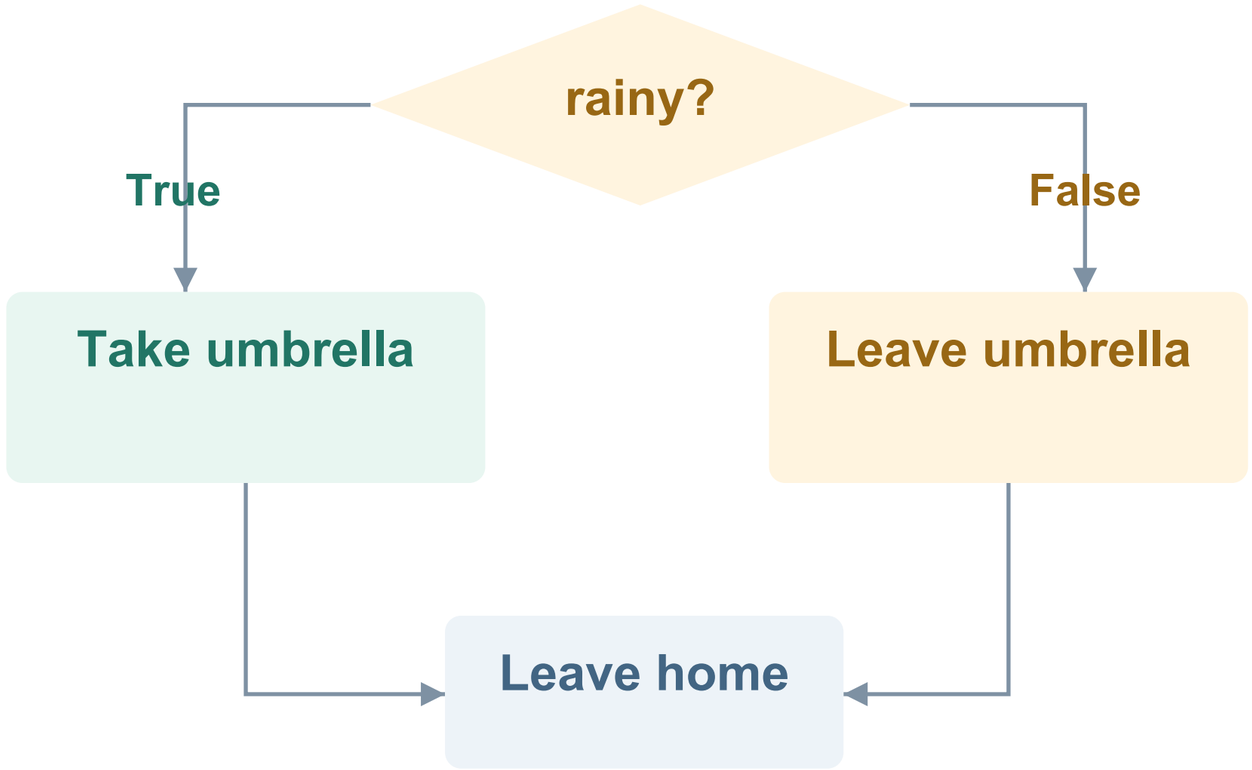
Choose a branch

The first matching branch runs. Indentation groups its statements.

```
count = 2
if count > 0: print("items")
```

One question, two possible paths

```
rainy = True
if rainy:
    print("Take umbrella")
else:
    print("Leave umbrella")
print("Leave home")
```



Only one indented block runs.

The unindented statement runs after either branch.

At the boundary, one symbol matters

A room with 20 chairs



`people = 20`

`people == 20`

Exactly 20

True

`people > 20`

More than 20

False

`people >= 20`

At least 20

True

= stores a value. == compares two values.

A comparison produces True or False. It does not choose a branch by itself.

Trace one selected branch

```
01 score = 20
02 enabled = True
03 print(score >= 20)
04 if enabled and score >= 20:
05     print("ready")
06 else:
07     print("waiting")
08 print("finished")
```

1. Expected: True, ready, finished.
2. Try score = 19, then enabled = False.
3. The final statement runs after either branch.

Comparisons produce Boolean values

```
01 total_portions = 20
02 reviewed = True
03 print(reviewed)
04 print(total_portions >= 20)
05 print(total_portions > 20)
06 print(reviewed and total_portions >= 20)
```

1. Predict each Boolean result.
2. >= includes equality; > excludes it.
3. Use Boolean values without quotes.

A comparison evaluates to True or False.

if / else: follow the selected block

```
reviewed = False
if reviewed:
    next_step = "inspect workload"
    print("Review recorded")
else:
    next_step = "coordinator follow-up"
    print("No review recorded")
print(next_step)
assert next_step == "coordinator follow-up"
```

1. Follow the path for False.
2. Both branches assign next_step.
3. Indentation defines each block.

A shared print runs after either branch.

if / elif / else: first matching branch

```
containers = 4
if containers == 0:
    draft_label = "empty"
elif containers < 4:
    draft_label = "small"
else:
    draft_label = "several"
print("Draft:", draft_label)
assert draft_label == "several"
```

1. Which branch handles four containers?
2. Trace zero → empty; three → small
3. Only the first matching branch runs

Order matters: zero matches empty before the small-count comparison.

and / or: compare the truth values

```
reviewed = False
total_portions = 21
print("AND:", reviewed and total_portions >= 20)
print("OR:", reviewed or total_portions >= 20)
assert (reviewed and total_portions >= 20) is False
assert (reviewed or total_portions >= 20) is True
```

1. Predict the two expressions.
2. False and True differs from False or True.
3. Change one input and compare again.

and requires both conditions; or requires at least one.

Boundary testing: > versus >=

```
01 total_portions = 20
02 buggy = total_portions > 20
03 policy = total_portions >= 20
04 print("Buggy:", buggy)
05 print("Policy:", policy)
06 assert buggy is False
07 assert policy is True
```

1. Which comparison agrees with the stated policy?
2. At 19 both comparisons are False
3. At 21 both comparisons are True

Keep the agreed expectation while diagnosing the implementation.

Hands-on practice

Week_02.ipynb · Session 2

Python practice

P1–P2 · Core

Two small programs with approach hints.

P3 · Core

A complete program with normal and edge-case tests.

P4–P5 · Optional

An application and a harder challenge.

P1–P3: your code, expected results and an explanation. P4–P5 are optional.

Project studio

Review-first dispatch decision

The project need

1

Need

The coordinator needs a rule for when a request can proceed.

2

Operation

Use the supplied review decision and total to choose hold, standard or team.

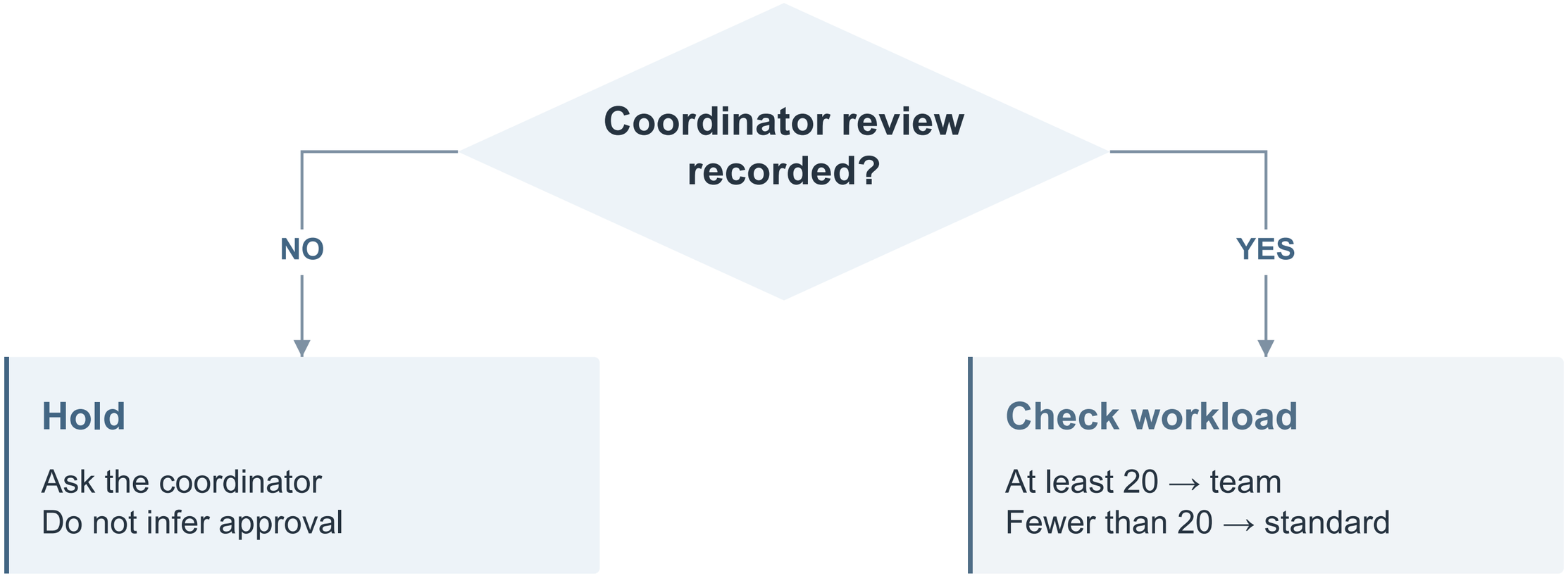
3

Result

The coordinator can see the label and the reason for it.

Final demo: keep human review before the workload rule.

Sketch the rule before implementing it



Review comes before the workload decision.

Project task

Supplied starting point

The notebook provides test inputs and a branching starter.

You run / explain

Implement review-first branching: unreviewed → hold; reviewed and total >= 20 → team; otherwise standard.

Check the evidence

Test 19, 20 and 21 with both review flags. Unreviewed always stays hold; exactly 20 is team when reviewed.

Unreviewed stays hold; reviewed totals of at least 20 use team.

Project checks

Step	What to show
Implementation	Implement hold / standard / team with review first.
Checks	Test 19, 20, 21 with review True and False.

Unreviewed stays hold; reviewed totals of at least 20 use team.

Further reading in English

1. ## Python tutorial: if statements

   https://docs.python.org/3/tutorial/controlflow.html#if-statements

2. ## Python Boolean operations

   https://docs.python.org/3/library/stdtypes.html#boolean-operations-and-or-not

Read one relevant section, then bring an example and a question.